Assignment No -9

Name - Payal Balasaheb More . **Roll No** - EN23107080 . **Class** - TY(B) . **Batch** - A

Title :Develop an AI model to analyze social media sentiment trends for financial markets.

Dataset-Financial Sentiment Analysis

import pandas as pd import numpy as np import matplotlib.pyplot as plt import tensorflow as tf from tensorflow.keras.preprocessing.text import Tokenizer from tensorflow.keras.preprocessing.sequence import pad_sequences from sklearn.model_selection import train_test_split from sklearn.preprocessing import LabelEncoder

```
df = pd.read_csv('data.csv')
```

```
le = LabelEncoder()
df['Sentiment_Label'] = le.fit_transform(df['Sentiment'])
num_classes = len(le.classes_)

# Tokenization & Padding
vocab_size = 10000
max_length = 64
tokenizer = Tokenizer(num_words=vocab_size, oov_token="<OOV>")
tokenizer.fit_on_texts(df['Sentence'])
```

```
X = tokenizer.texts_to_sequences(df['Sentence'])
X_padded = pad_sequences(X, maxlen=max_length, padding='post', truncating='post')
y = df['Sentiment_Label']
```

```
X_train, X_test, y_train, y_test = train_test_split(X_padded, y, test_size=0.2, random_state=42)

# --- MODEL 1: Long Short-Term Memory (LSTM) ---
# Good for understanding word order and long-range dependencies in finance
model_lstm = tf.keras.Sequential([
    tf.keras.layers.Embedding(vocab_size, 32, input_length=max_length),
    tf.keras.layers.LSTM(64, dropout=0.2, recurrent_dropout=0.2),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(num_classes, activation='softmax')
])
model_lstm.compile(loss='sparse_categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
```

```
model_cnn = tf.keras.Sequential([
    tf.keras.layers.Embedding(vocab_size, 32, input_length=max_length),
    tf.keras.layers.Conv1D(128, 5, activation='relu'),
    tf.keras.layers.GlobalMaxPooling1D(),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(num_classes, activation='softmax')
])
model_cnn.compile(loss='sparse_categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
```

```
print("Training LSTM Model...")
history_lstm = model_lstm.fit(X_train, y_train, epochs=8, validation_data=(X_test, y_test), verbose=0)

print("Training CNN Model...")
history_cnn = model_cnn.fit(X_train, y_train, epochs=8, validation_data=(X_test, y_test), verbose=0)
```

```
Training LSTM Model...
Training CNN Model...
```

```
print("Training LSTM Model...")
history_lstm = model_lstm.fit(X_train, y_train, epochs=8, validation_data=(X_test, y_test), verbose=0)

print("Training CNN Model...")
history_cnn = model_cnn.fit(X_train, y_train, epochs=8, validation_data=(X_test, y_test), verbose=0)

# 4. Comparison Visualization
lstm_acc = history_lstm.history['val_accuracy'][-1]
cnn_acc = history_cnn.history['val_accuracy'][-1]
```

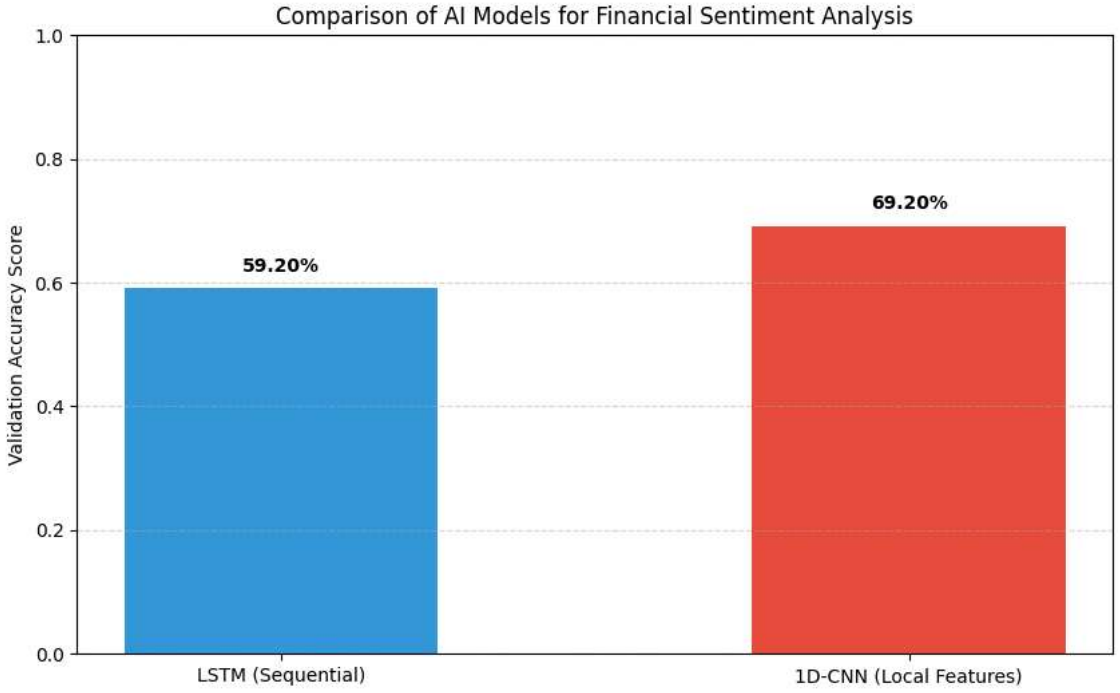
```
models = ['LSTM (Sequential)', '1D-CNN (Local Features)']
accuracies = [lstm_acc, cnn_acc]

plt.figure(figsize=(10, 6))
bars = plt.bar(models, accuracies, color=['#3498db', '#e74c3c'], width=0.5)
plt.ylabel('Validation Accuracy Score')
plt.title('Comparison of AI Models for Financial Sentiment Analysis')
plt.ylim(0, 1.0)
plt.grid(axis='y', linestyle='--', alpha=0.5)

# Adding labels on top of bars
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval + 0.02, f'{yval:.2%}', ha='center', va='bottom', fontweight='bold')

plt.savefig('comparison_plot.png')
```

```
Training LSTM Model...
Training CNN Model...
```



Start coding or generate with AI.